Academic Year: 2026-27 Sap Id: 60003240292

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

COURSE CODE: DJS23ICPC501 **DATE: 31/07/2026**

COURSE NAME: Computer Networks **CLASS: T. Y. B.Tech**

Experiment No. 2

CO/LO: Analyse various concerns at different layers in reference models using suitable techniques.

Aim: Write a program to implement Framing Techniques: Character count, Byte stuffing, Bit stuffing

Implement Framing

EXERCISE

Your program will read a file (in pdf). Convert given files into ASCII and then Binary values whichever applicable. Use this prepared file and create frames of manageable units called frames with following three options

1. Character count: Randomly generate character count between 2^0 to 2^16. Display the various frames created with the help of preprocessed file as above.
2. Bit Stuffing: Generate Fixed frames of equal length as per used defined input probably Power of 2(2^n) from above preprocessed file by using bit stuffing method.
   Assume (01111110) as Start and End of Frame. Bit "Zero" should be stuffed in payload if 5 consecutive "1" occurs in data. Pad your data sequence (1000000…..0) to make the preprocessed file multiple of 128.
3. Byte Stuffing: Implement variable length byte stuffing using character stuffing method. Insert special character ("ESC") character in preprocessed file at random interval between 2^0 to 2^16. Display the various frames created with the help of preprocessed file as above.

CODE:

```
import random

with open("abc.txt") as file:
    data = file.read()

asci = list(map(ord, data))
binary = "".join(format(x, "08b") for x in asci)
```

```
FLAG = "01111110"
ESC = "10101010"

def character_count(data):
    i = 0
    n = len(data)
    frames = []
    frame_count = 0
    while i < n:
        count = random.randint(1, 16)
        actual = min(count, n - i)
        frames.append(actual)
        frames.extend(data[i:i + actual])
        i += actual
        frame_count += 1
    overhead = frame_count * 8
    return frames, overhead

def bit_stuffing(data, frame_size):
    i = 0
    frames = []
    count1 = 0
    stuffed = 0
    flag_count = 0
    while i < len(data):
        if i % frame_size == 0:
            frames.append(f"|{FLAG}|")
            flag_count += 1
            if i != 0:
                frames.append(f"|{FLAG}|")
                flag_count += 1
        frames.append(data[i])
        if data[i] == '1':
            count1 += 1
            if count1 == 5:
                frames.append('0')
                stuffed += 1
                count1 = 0
        else:
            count1 = 0
        i += 1
    padding = 0
    if (len(data) + stuffed) % frame_size != 0:
        padding = frame_size - (len(data) + stuffed) % frame_size
        for i in range(padding):
            frames.append("0")
    frames.append(f"|{FLAG}|")
```

```
    flag_count += 1
    overhead = stuffed + padding + flag_count * 8
    return "".join(frames), overhead

def byte_stuffing(data, frame_size):
    frames = []
    byts = [data[i:i + 8] for i in range(0, len(data), 8)]
    if len(byts[-1]) != 8:
        byts[-1] += "0" * (8 - len(byts[-1]))
    i = 0
    flag_count = 0
    esc_count = 0
    while i < len(byts):
        frames.append(f"|{FLAG}|")
        flag_count += 1
        end = min(i + frame_size, len(byts))
        for j in range(i, end):
            if byts[j] == FLAG or byts[j] == ESC:
                frames.append(ESC)
                esc_count += 1
            frames.append(byts[j])
        i = end
    overhead = (flag_count + esc_count) * 8
    return "".join(frames), overhead

if __name__ == "__main__":
    data = binary

    while True:
        print("\nMethod:")
        print("1. Character Count")
        print("2. Byte Stuffing")
        print("3. Bit Stuffing")
        print("4. Exit")

        q = input("Enter: ")

        if q == "1":
            result, overhead = character_count(data)
            print(result)
            print("Overhead Percentage: {:.2f}%".format((overhead / len(data)) * 100))

        elif q == "2":
            frame_size = int(input("Enter frame size: "))
            result, overhead = byte_stuffing(data, frame_size)
            print(result)
            print("Overhead Percentage: {:.2f}%".format((overhead / len(data)) * 100))
```

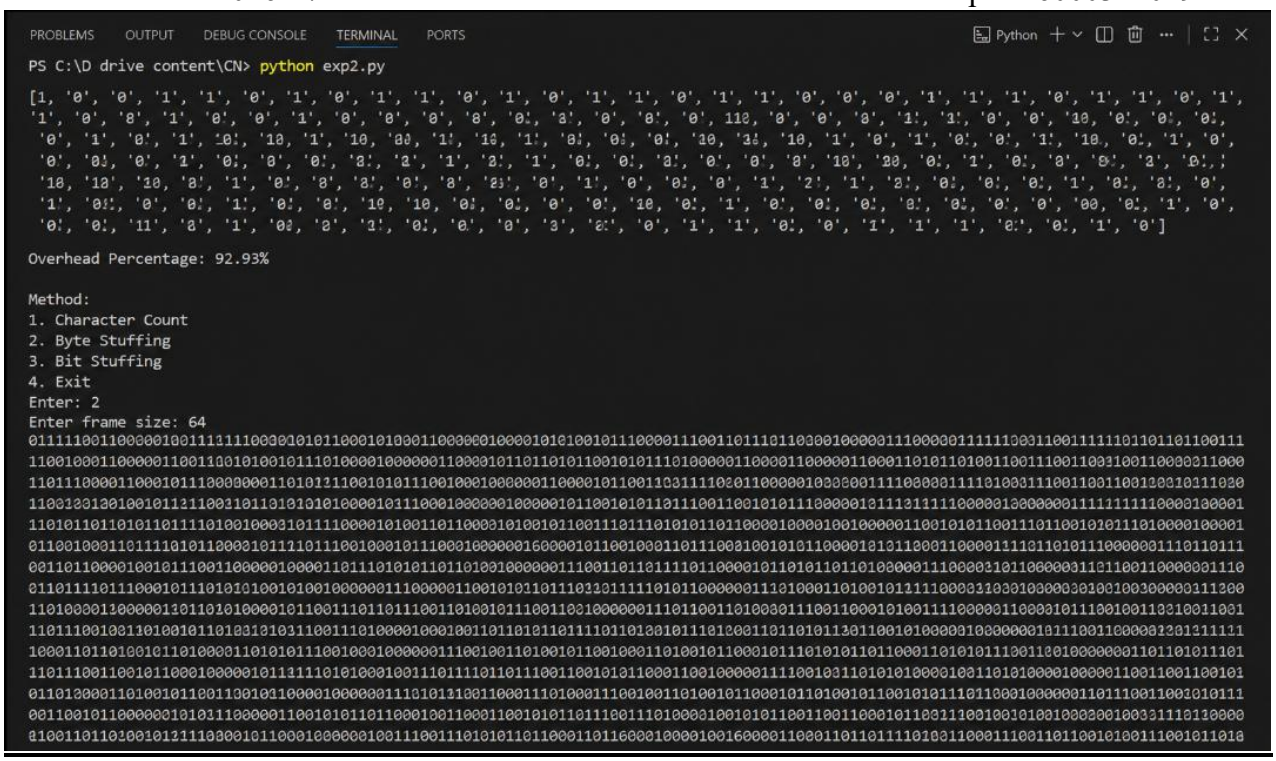
```
    elif q == "3":
        frame_size = int(input("Enter frame size: "))
        result, overhead = bit_stuffing(data, frame_size)
        print(result)
        print("Overhead Percentage: {:.2f}%".format((overhead / len(data)) * 100))

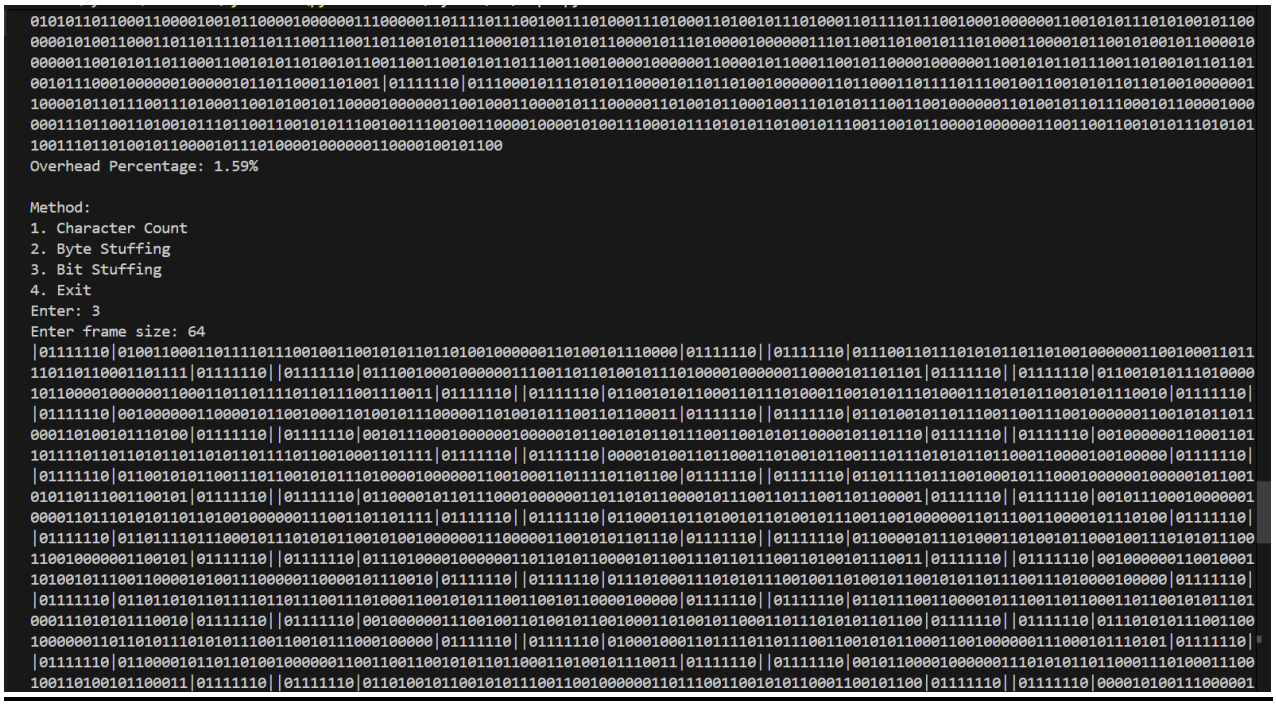
    elif q == "4":
        break

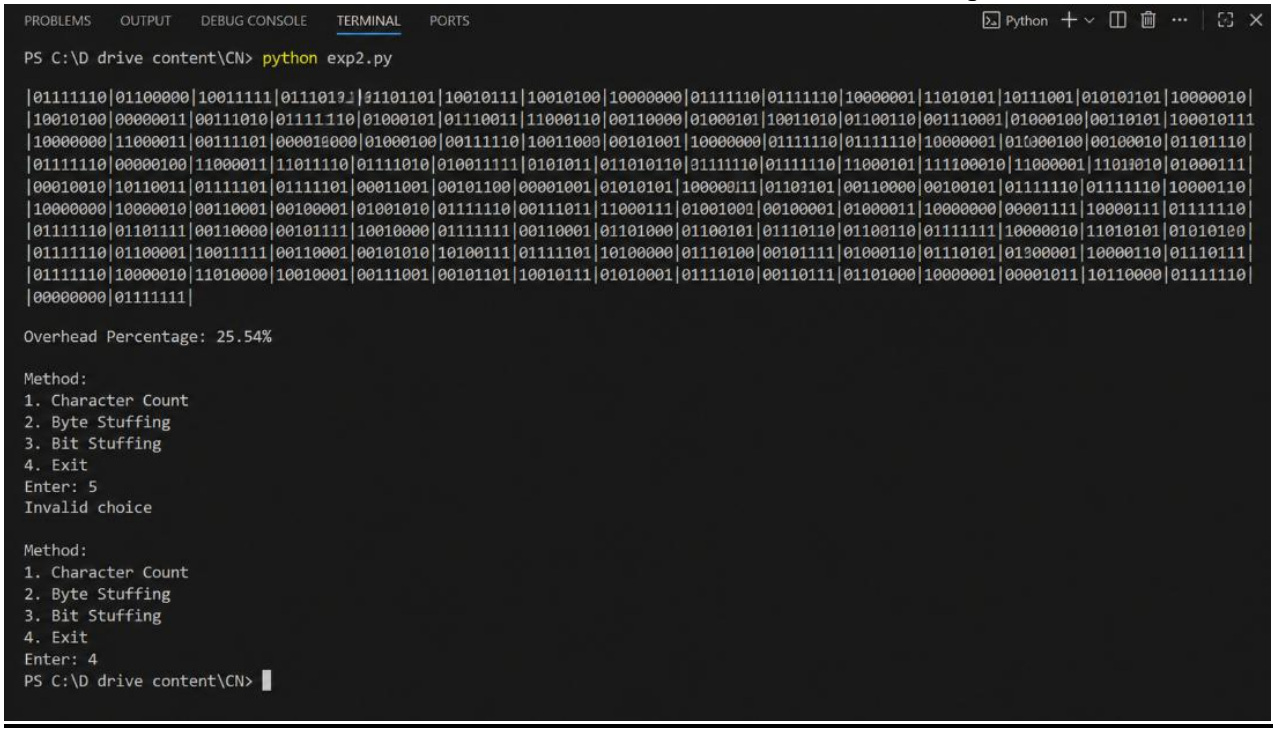
    else:
        print("Invalid choice")
```

OUTPUT:




PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
PS C:\D drive content\CN> python exp2.py
Overhead Percentage: 92.93%
Method:
1. Character Count
2. Byte Stuffing
3. Bit Stuffing
4. Exit
Enter: 2
Enter frame size: 64


Overhead Percentage: 1.59%
Method:
1. Character Count
2. Byte Stuffing
3. Bit Stuffing
4. Exit
Enter: 3
Enter frame size: 64



Observation:

Sr. No	Frame Size	No. of Frames (Bit Stuffing | Byte Stuffing)	Overhead (%) (Bit Stuffing | Byte Stuffing)
1	64	11 | 87	1.59 | 25.54
2	128	6 | 44	0.87 | 14.29
3	256	3 | 22	0.43 | 7.94

Conclusion: Study of network commands helps to understand the network configuration better

Reference:

[1] www.geeksforgeeks.org